1. Data Definition Language (DDL)

Database Operations

```
-- Create database
create database database_name;

-- Drop (delete) database
drop database database_name;

-- Show all databases
show databases;

-- Use a database
use database_name;
```

Table Operations

```
-- Create table
create table table_name (
    column1 datatype constraints,
    column2 datatype constraints
);

-- Drop (delete) table
drop table table_name;

-- Show all tables
show tables;

-- Show table structure
describe table_name;
-- or
desc table_name;
-- or
show columns from table_name;

-- Truncate (delete all data, keep structure)
truncate table table_name;

-- Rename table
rename table old_name to new_name;
-- or
```

```
alter table old_name rename to new_name;
```

Alter Table Operations

```
-- Add column
alter table table_name add column_name datatype;

-- Drop column
alter table table_name drop column column_name;

-- Modify column (change datatype)
alter table table_name modify column column_name new_datatype;

-- Change column (rename + change datatype)
alter table table_name change old_name new_name datatype;

-- Rename column (MySQL 8.0+)
alter table table_name rename column old_name to new_name;

-- Add primary key
alter table table_name add primary key (column_name);

-- Drop primary key
alter table table_name drop primary key;

-- Add foreign key
alter table table_name add foreign key (column_name)
references other_table(column_name);

-- Add unique constraint
alter table table_name add unique (column_name);

-- Add check constraint
alter table table_name add check (condition);

-- Set default value
alter table table_name alter column_name set default value;

-- Drop default value
alter table table_name alter column_name drop default;
```

---

2. Data Manipulation Language (DML)

Insert Data

```
-- Insert single row
insert into table_name values (value1, value2, ...);

-- Insert with column names
insert into table_name (col1, col2) values (val1, val2);

-- Insert multiple rows
insert into table_name values
(val1, val2),
(val3, val4),
(val5, val6);

-- Insert from another table
insert into table1 (col1, col2)
select col1, col2 from table2 where condition;
```

Update Data

```
-- Update specific rows
update table_name set column1 = value1 where condition;

-- Update multiple columns
update table_name
set col1 = val1, col2 = val2
where condition;

-- Update all rows (dangerous!)
update table_name set column1 = value1;

-- Update with calculation
update table_name set column1 = column1 + 10;

-- Update using subquery
update table_name set column1 = (select value from table2 where condition);
```

Delete Data

```
-- Delete specific rows
delete from table_name where condition;
```

```
-- Delete all rows (dangerous!)
delete from table_name;

-- Delete using subquery
delete from table_name where column in (select column from table2);
```

---

3. Data Query Language (DQL)

Basic SELECT

```
-- Select all columns
select * from table_name;

-- Select specific columns
select col1, col2 from table_name;

-- Select with alias
select col1 as alias_name from table_name;

-- Select distinct values
select distinct column_name from table_name;

-- Select with limit
select * from table_name limit 10;

-- Select with offset
select * from table_name limit 10 offset 5;
```

WHERE Clause (Filtering)

```
-- Basic conditions
select * from table_name where column = value;
select * from table_name where column > value;
select * from table_name where column < value;
select * from table_name where column >= value;
select * from table_name where column <= value;
select * from table_name where column != value;
select * from table_name where column <> value;  -- same as !=

-- Multiple conditions
select * from table_name where col1 = val1 and col2 = val2;
```

```
select * from table_name where col1 = val1 or col2 = val2;
select * from table_name where not column = value;

-- NULL checks
select * from table_name where column is null;
select * from table_name where column is not null;

-- Range
select * from table_name where column between val1 and val2;
select * from table_name where column not between val1 and val2;

-- List of values
select * from table_name where column in (val1, val2, val3);
select * from table_name where column not in (val1, val2, val3);
```

Pattern Matching (LIKE)

```
-- Starts with
select * from table_name where column like 'A%';

-- Ends with
select * from table_name where column like '%z';

-- Contains
select * from table_name where column like '%abc%';

-- Not like
select * from table_name where column not like '%abc%';

-- Underscore (single character)
select * from table_name where column like '_a%';

-- Case-insensitive (default in MySQL)
select * from table_name where column like 'john';

-- Regular expressions (MySQL)
select * from table_name where column regexp 'pattern';
select * from table_name where column rlike 'pattern';
```

ORDER BY (Sorting)

```
-- Ascending order (default)
select * from table_name order by column asc;
```

```
select * from table_name order by column;  -- ASC is default

-- Descending order
select * from table_name order by column desc;

-- Multiple columns
select * from table_name order by col1 asc, col2 desc;

-- Order by expression
select * from table_name order by (col1 + col2) desc;

-- Order by alias
select col1, col2 as total from table_name order by total;

-- Order by position (not recommended)
select col1, col2 from table_name order by 2;  -- orders by 2nd column
```

---

4. Aggregate Functions

Basic Aggregates

```
-- Count rows
select count(*) from table_name;
select count(column_name) from table_name;  -- excludes NULL
select count(distinct column_name) from table_name;

-- Sum
select sum(column_name) from table_name;

-- Average
select avg(column_name) from table_name;

-- Minimum
select min(column_name) from table_name;

-- Maximum
select max(column_name) from table_name;
```

GROUP BY

```
-- Basic grouping
select column, count(*) from table_name group by column;
```

```
-- Multiple columns
select col1, col2, count(*) from table_name group by col1, col2;

-- With aggregate functions
select major, avg(cgpa), max(marks) from students group by major;

-- HAVING (filter groups)
select column, count(*) from table_name
group by column
having count(*) > 5;

-- GROUP BY with WHERE and HAVING
select column, avg(value) from table_name
where condition
group by column
having avg(value) > 100;

-- GROUP BY with ROLLUP (subtotals)
select col1, col2, sum(col3) from table_name
group by col1, col2 with rollup;
```

---

5. String Functions

```
-- Uppercase and lowercase
select upper(column_name) from table_name;
select lower(column_name) from table_name;

-- Length
select length(column_name) from table_name;
select char_length(column_name) from table_name;

-- Concatenate
select concat(col1, ' ', col2) from table_name;
select concat_ws('-', col1, col2, col3) from table_name;  -- with separator

-- Substring
select substring(column_name, start, length) from table_name;
select substr(column_name, start, length) from table_name;
select left(column_name, 5) from table_name;  -- first 5 chars
select right(column_name, 5) from table_name;  -- last 5 chars
```

```
-- Trim whitespace
select trim(column_name) from table_name;
select ltrim(column_name) from table_name;  -- left trim
select rtrim(column_name) from table_name;  -- right trim

-- Replace
select replace(column_name, 'old', 'new') from table_name;

-- Reverse
select reverse(column_name) from table_name;

-- Repeat
select repeat(column_name, 3) from table_name;

-- Position
select position('text' in column_name) from table_name;
select locate('text', column_name) from table_name;
select instr(column_name, 'text') from table_name;
```

---

6. Numeric Functions

```
-- Rounding
select round(column_name, 2) from table_name;  -- 2 decimal places
select ceiling(column_name) from table_name;  -- round up
select floor(column_name) from table_name;  -- round down
select truncate(column_name, 2) from table_name;  -- truncate

-- Absolute value
select abs(column_name) from table_name;

-- Power and square root
select power(column_name, 2) from table_name;
select sqrt(column_name) from table_name;

-- Random
select rand() from table_name;
select rand() * 100 from table_name;  -- random 0-100

-- Modulo (remainder)
select mod(column_name, 3) from table_name;
select column_name % 3 from table_name;
```

```
-- Sign
select sign(column_name) from table_name;  -- returns -1, 0, or 1
```

---

7. Date and Time Functions

```
-- Current date and time
select now();  -- current datetime
select curdate();  -- current date
select curtime();  -- current time
select current_timestamp();

-- Extract parts
select year(date_column) from table_name;
select month(date_column) from table_name;
select day(date_column) from table_name;
select hour(datetime_column) from table_name;
select minute(datetime_column) from table_name;
select second(datetime_column) from table_name;

-- Day of week/year
select dayofweek(date_column) from table_name;  -- 1=Sunday
select dayofmonth(date_column) from table_name;
select dayofyear(date_column) from table_name;
select weekday(date_column) from table_name;  -- 0=Monday

-- Week and quarter
select week(date_column) from table_name;
select quarter(date_column) from table_name;

-- Month and day names
select monthname(date_column) from table_name;
select dayname(date_column) from table_name;

-- Date arithmetic
select date_add(date_column, interval 7 day) from table_name;
select date_sub(date_column, interval 1 month) from table_name;
select adddate(date_column, 7) from table_name;
select subdate(date_column, 7) from table_name;

-- Date difference
select datediff(date1, date2) from table_name;
select timediff(time1, time2) from table_name;
```

```
-- Formatting
select date_format(date_column, '%Y-%m-%d') from table_name;
select time_format(time_column, '%H:%i:%s') from table_name;

-- Convert to date/time
select date(datetime_column) from table_name;
select time(datetime_column) from table_name;

-- Last day of month
select last_day(date_column) from table_name;
```

---

8. Conditional Functions

```
-- IF statement
select if(condition, true_value, false_value) from table_name;
select if(cgpa > 3.5, 'Good', 'Average') from students;

-- CASE statement
select
   column_name,
   case
      when condition1 then result1
      when condition2 then result2
      else result3
   end as alias_name
from table_name;

-- Example
select
   name,
   case
      when cgpa >= 3.7 then 'Excellent'
      when cgpa >= 3.3 then 'Good'
      when cgpa >= 3.0 then 'Average'
      else 'Poor'
   end as grade
from students;

-- IFNULL (handle null values)
select ifnull(column_name, 'default_value') from table_name;
```

```
-- NULLIF (return null if equal)
select nullif(col1, col2) from table_name;

-- COALESCE (return first non-null)
select coalesce(col1, col2, col3, 'default') from table_name;
```

---

9. Subqueries

```
-- Scalar subquery (returns single value)
select * from table_name
where column = (select max(column) from table_name);

-- Multiple row subquery
select * from table_name
where column in (select column from other_table);

-- Correlated subquery
select * from table1 t1
where column > (select avg(column) from table1 t2 where t2.category = t1.category);

-- Subquery in SELECT
select name, (select count(*) from orders where orders.customer_id = customers.id)
from customers;

-- Subquery in FROM (derived table)
select avg(total) from (select sum(amount) as total from sales group by id) as subquery;

-- EXISTS
select * from table1
where exists (select 1 from table2 where table2.id = table1.id);

-- NOT EXISTS
select * from table1
where not exists (select 1 from table2 where table2.id = table1.id);

-- ANY (at least one)
select * from table_name
where column > any (select column from other_table);

-- ALL (all values)
select * from table_name
where column > all (select column from other_table);
```

```
-- SOME (synonym for ANY)
select * from table_name
where column > some (select column from other_table);
```

---

10. Joins

```
-- INNER JOIN (only matching rows)
select * from table1
inner join table2 on table1.id = table2.id;

-- LEFT JOIN (all from left, matching from right)
select * from table1
left join table2 on table1.id = table2.id;

-- RIGHT JOIN (all from right, matching from left)
select * from table1
right join table2 on table1.id = table2.id;

-- FULL OUTER JOIN (all rows from both) - MySQL workaround
select * from table1 left join table2 on table1.id = table2.id
union
select * from table1 right join table2 on table1.id = table2.id;

-- CROSS JOIN (cartesian product)
select * from table1 cross join table2;
select * from table1, table2;  -- implicit cross join

-- SELF JOIN
select a.name, b.name from employees a
join employees b on a.manager_id = b.id;

-- Multiple joins
select * from table1
join table2 on table1.id = table2.id
join table3 on table2.id = table3.id;

-- Join with WHERE
select * from table1
join table2 on table1.id = table2.id
where table1.column = 'value';
```

---

11. Set Operations

```
-- UNION (combine results, remove duplicates)
select column from table1
union
select column from table2;

-- UNION ALL (combine results, keep duplicates)
select column from table1
union all
select column from table2;

-- INTERSECT (common rows) - MySQL 8.0.31+
select column from table1
intersect
select column from table2;

-- EXCEPT / MINUS (rows in first but not second) - MySQL 8.0.31+
select column from table1
except
select column from table2;
```

---

12. Window Functions (MySQL 8.0+)

```
-- ROW_NUMBER (assign row numbers)
select name, row_number() over (order by cgpa desc) as rank
from students;

-- RANK (with gaps for ties)
select name, rank() over (order by cgpa desc) as rank
from students;

-- DENSE_RANK (no gaps for ties)
select name, dense_rank() over (order by cgpa desc) as rank
from students;

-- PARTITION BY (separate rankings per group)
select name, major, rank() over (partition by major order by cgpa desc) as rank
from students;
```

```
-- LAG (previous row value)
select name, cgpa, lag(cgpa) over (order by cgpa) as prev_cgpa
from students;

-- LEAD (next row value)
select name, cgpa, lead(cgpa) over (order by cgpa) as next_cgpa
from students;

-- FIRST_VALUE and LAST_VALUE
select name, first_value(name) over (order by cgpa desc) as top_student
from students;

-- NTILE (divide into groups)
select name, ntile(4) over (order by cgpa) as quartile
from students;
```

---

13. Views

```
-- Create view
create view view_name as
select col1, col2 from table_name where condition;

-- Use view
select * from view_name;

-- Update view
create or replace view view_name as
select col1, col2 from table_name where new_condition;

-- Drop view
drop view view_name;

-- Show views
show full tables where table_type = 'VIEW';
```

---

14. Indexes

```
-- Create index
create index index_name on table_name (column_name);
```

```
-- Create unique index
create unique index index_name on table_name (column_name);

-- Create composite index
create index index_name on table_name (col1, col2);

-- Drop index
drop index index_name on table_name;

-- Show indexes
show indexes from table_name;
```

---

15. Constraints

```
-- Primary key
create table table_name (
    id int primary key,
    name varchar(50)
);

-- Foreign key
create table orders (
    id int primary key,
    customer_id int,
    foreign key (customer_id) references customers(id)
);

-- Unique
create table table_name (
    email varchar(100) unique
);

-- Not null
create table table_name (
    name varchar(50) not null
);

-- Check
create table table_name (
    age int check (age >= 18)
);
```

```
-- Default
create table table_name (
    status varchar(20) default 'active'
);

-- Auto increment
create table table_name (
    id int auto_increment primary key
);
```

---

16. Transactions (Data Control Language)

```
-- Start transaction
start transaction;
-- or
begin;

-- Commit (save changes)
commit;

-- Rollback (undo changes)
rollback;

-- Savepoint
savepoint savepoint_name;

-- Rollback to savepoint
rollback to savepoint savepoint_name;

-- Set autocommit
set autocommit = 0;  -- disable
set autocommit = 1;  -- enable
```

---

17. User Management

```
-- Create user
create user 'username'@'localhost' identified by 'password';

-- Grant privileges
grant all privileges on database_name.* to 'username'@'localhost';
```

```
grant select, insert on database_name.table_name to 'username'@'localhost';

-- Revoke privileges
revoke all privileges on database_name.* from 'username'@'localhost';

-- Show grants
show grants for 'username'@'localhost';

-- Drop user
drop user 'username'@'localhost';

-- Change password
alter user 'username'@'localhost' identified by 'new_password';

-- Flush privileges
flush privileges;
```

---

18. Information Schema

```
-- Show table information
select * from information_schema.tables
where table_schema = 'database_name';

-- Show column information
select * from information_schema.columns
where table_name = 'table_name';

-- Show constraints
select * from information_schema.table_constraints
where table_name = 'table_name';

-- Database size
select
    table_schema as 'Database',
    sum(data_length + index_length) / 1024 / 1024 as 'Size (MB)'
from information_schema.tables
group by table_schema;
```

---

19. Other Useful Commands

```
-- Show warnings
show warnings;

-- Show errors
show errors;

-- Explain query execution plan
explain select * from table_name where condition;

-- Show processlist
show processlist;

-- Kill process
kill process_id;

-- Show variables
show variables;
show variables like 'max_connections';

-- Set variable
set global max_connections = 200;

-- Show status
show status;
show status like 'Threads_connected';

-- Optimize table
optimize table table_name;

-- Analyze table
analyze table table_name;

-- Check table
check table table_name;

-- Repair table
repair table table_name;
```

---

20. Import/Export

```
-- Export to CSV
select * from table_name
```

```
into outfile '/path/to/file.csv'
fields terminated by ','
enclosed by '"'
lines terminated by '\n';

-- Import from CSV
load data infile '/path/to/file.csv'
into table table_name
fields terminated by ','
enclosed by '"'
lines terminated by '\n';

-- mysqldump (command line)
-- mysqldump -u username -p database_name > backup.sql

-- Restore (command line)
-- mysql -u username -p database_name < backup.sql
```

Query Execution Order

Understanding the order SQL processes queries:

1. **FROM** - Choose tables
2. **JOIN** - Combine tables
3. **WHERE** - Filter rows
4. **GROUP BY** - Group rows
5. **HAVING** - Filter groups
6. **SELECT** - Choose columns
7. **DISTINCT** - Remove duplicates
8. **ORDER BY** - Sort results
9. **LIMIT** - Limit results

Data Types Reference

Numeric

- `INT`, `INTEGER` - Integer
- `SMALLINT` - Small integer
- `BIGINT` - Large integer

- `DECIMAL(p,s)` - Fixed-point decimal
- `NUMERIC(p,s)` - Same as DECIMAL
- `FLOAT` - Floating-point
- `DOUBLE` - Double precision float
- `BIT` - Bit value

String

- `CHAR(n)` - Fixed-length string
- `VARCHAR(n)` - Variable-length string
- `TEXT` - Long text
- `TINYTEXT` - Very small text
- `MEDIUMTEXT` - Medium text
- `LONGTEXT` - Very long text
- `BLOB` - Binary large object
- `ENUM('val1','val2')` - Enumeration
- `SET('val1','val2')` - Set of values

Date/Time

- `DATE` - Date (YYYY-MM-DD)
- `TIME` - Time (HH:MM:SS)
- `DATETIME` - Date and time
- `TIMESTAMP` - Timestamp
- `YEAR` - Year

Other

- `BOOLEAN` / `BOOL` - True/False (TINYINT(1))
- `JSON` - JSON data (MySQL 5.7+)